

习题课 5

问题 5-1 图的半径

在任意无向图 $G = (V, E)$ 中，顶点 $u \in V$ 的**离心率** $e(u)$ 是从 u 到距其最远顶点 v 的最短距离，即

$$e(u) = \max\{\delta(u, v) \mid v \in V\}.$$

无向图 $G = (V, E)$ 的**半径** $R(G)$ 是所有顶点离心率中的最小值，即

$$R(G) = \min\{e(u) \mid u \in V\}.$$

- (a) 给定一个连通无向图 G ，描述一个运行时间为 $O(|V||E|)$ 的算法来确定 G 的半径。
- (b) 给定一个连通无向图 G ，描述一个运行时间为 $O(|E|)$ 的算法，求出 G 的半径的一个上界 R^* ，使得

$$R(G) \leq R^* \leq 2R(G).$$

问题 5-2 互联网调查

麻省理工学院收到了一些关于其 WiFi 网络速度的投诉。该网络由 r 台路由器组成，其中一些被标记为连接到互联网其余部分的**入口点**。某些路由器对之间通过双向线路直接相连。路由器之间的每条线路 w_i 都具有一个已知长度 ℓ_i ，该长度用正整数英尺数表示。

网络中一台路由器的**延迟**，与信号从该路由器到达某个入口点所必须经过的线路的最小总长度成正比。假设每台路由器的延迟都是有限的，并且整个网络中的线路总长度至多为 $100r$ 英尺。

给定一张网络示意图，其中标出了所有路由器和线路长度，请描述一个运行时间为 $O(r)$ 的算法，计算网络中所有路由器的延迟之和。

问题 5-3 四巫师探索

女巫波特里·哈特（Potry Harter）和她的三位巫师朋友接到任务，要搜索一座迷宫中的每一个房间，以寻找魔法物品。这座迷宫由 n 个房间组成，每个房间至多有四扇通往其他房间的门。假设开始时所有门都是关闭的，并且从一个指定的入口房间出发，可以到达迷宫中的每一个房间。

有些门受到邪恶魔法的保护，必须先被**解除魔法**才能打开；其他门则可以直接打开。给定一张迷宫地图，其中标明了每扇门是否被施加了魔法，请描述一个运行时间为 $O(n)$ 的算法，确定从入口房间开始，为访问迷宫中的每一个房间而必须解除魔法的最少门数。

问题 5-4 纯净大西洋

布里查德·兰森 (Brichard Ranson) 是国际旅行公司“纯净大西洋” (Purity Atlantic) 的创始人, 该公司专门为新婚夫妇策划豪华蜜月旅行。预订定制旅行时, 一对夫妇需要提交他们的居住城市, 以及希望在蜜月期间游览的三个城市。随后, 纯净大西洋会安排包括飞行行程在内的所有事项: 飞行行程是一个航班序列, 从夫妇的居住城市出发, 以任意顺序依次前往三个游览城市, 最后返回居住城市。

遗憾的是, 任意两个城市之间并不总有直飞航班, 因此可能需要搭乘多段航班。尽管费用和时间并非需要考虑的因素, 但夫妇们希望尽量减少蜜月期间所搭乘的直飞航班总数。

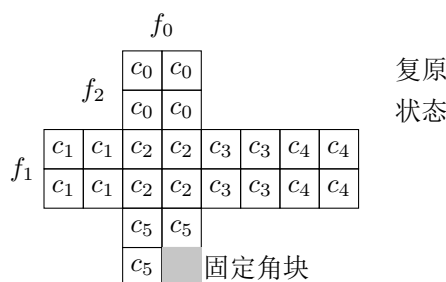
给定一份包含 c 个城市的列表和一份包含全部 f 个可用直飞航班的列表, 其中每个直飞航班由一个有序城市对 (出发地、目的地) 表示, 请描述一个高效算法, 为给定夫妇确定一条使其搭乘直飞航班数量最少的飞行行程。

问题 5-5 口袋魔方

口袋魔方 (Pocket Cube) 是传统 $3 \times 3 \times 3$ 魔方的一种较小的 $2 \times 2 \times 2$ 变体, 由八个角块组成; 每个角块在三个可见面上各有一种不同的颜色。魔方的复原状态是指口袋魔方的每个 2×2 面均为单色。我们用下标 $i \in \{0, \dots, 5\}$ 表示每种颜色 c_i 。

不失一般性, 我们固定其中一个角块的位置和朝向, 并且只允许绕不包含该固定角块的三个面 $\{f_0, f_1, f_2\}$ 的法向量进行单次旋转。具体而言, 一次操作由二元组 (j, s) 表示, 对应于将面 f_j 旋转一次: 当 $s = 1$ 时顺时针旋转, 当 $s = -1$ 时逆时针旋转。

可以使用广度优先搜索来求解口袋魔方: 搜索一张图, 其顶点为魔方所有可能的状态; 如果一个状态可以通过一次操作变为另一个状态, 则在两个状态之间连一条边。无需显式保存这些邻接关系, 只需对给定状态应用所有可能的单次操作, 即可计算其所有邻居。



- (a) 说明口袋魔方不同状态的数量少于 1200 万 (尝试利用组合数学给出尽可能紧的上界)。
- (b) 写出口袋魔方状态图中任意顶点的最大度数和最小度数。
- (c) 习题模板中给出的代码会从一个给定状态开始, 使用广度优先搜索完整探索口袋魔方的状态图, 然后返回一系列能够复原口袋魔方的操作 (假设复原状态可达)。然而, 这个求解器运行得非常缓慢。¹运行所提供的代码, 并写出搜索所访问的状态数。这个数与你在第 (a) 问中得到的上界相比如何?

¹请注意, 代码需要几分钟和相当多的内存 (超过 400 MB) 才能运行完毕。

- (d) 写出复原任意一个可解口袋魔方所需操作数的最大值 w 。
- (e) 对于某个特定状态，令 N_i 表示在 i 次操作之内可达的口袋魔方状态数。所提供的代码会访问 N_w 个状态（超过 300 万个）。请描述一个算法，为任意口袋魔方状态找出一条最短复原操作序列，或者返回不存在这样的序列；该算法访问的状态数不超过

$$2N_{\lceil w/2 \rceil},$$

而这一数量少于 9 万个。

- (f) 根据第 (e) 问中的算法，重写所提供模板代码中的 `solve(config)` 函数。

问题 5-5 附录：口袋魔方代码模板

下面是本题提供的 Python 模板代码。第(f)问只需根据第(e)问中的算法,重写其中的 `solve(config)` 函数；分隔线以下的代码无需修改。

```

1  # ----- #
2  # REWRITE SOLVE IMPLEMENTING PART (e) #
3  # ----- #
4  def solve(config):
5      # Return a sequence of moves to solve config, or None if not possible
6
7      # Fully explore graph using BFS
8      parent, frontier = {config: None}, [config]
9      while len(frontier) != 0:
10         frontier = explore_frontier(frontier, parent, True)
11         print('Searched %s reachable configurations' % len(parent))
12
13     # Check whether solved state visited and reconstruct path
14     if SOLVED in parent:
15         path = path_to_config(SOLVED, parent)
16         return moves_from_path(path)
17     return None
18
19 # ----- #
20 # READ, BUT DO NOT MODIFY CODE BELOW HERE
21 # ----- #
22 # Pocket Cube configurations are represented by length 24 strings
23 # Each character represents the color of a small cube face
24 # Faces are layed out in reading order of a Latin cross unfolding of the cube
25
26 SOLVED = '000011223344112233445555'
27
28 def config_str(config):
29     # Return config string representation as a Latin cross unfolding
30     return """
31         %s%s
32         %s%s
33     %s%s%s%s%s%s%s%s
34     %s%s%s%s%s%s%s%s
35         %s%s
36         %s%s
37     """ % tuple(config)
38
39 def shift(A, d, ps):
40     # Circularly shift values at indices ps in list A by d positions
41     values = [A[p] for p in ps]

```

```

42     k = len(ps)
43     for i in range(k):
44         A[ps[i]] = values[(i - d) % k]
45
46 def rotate(config, face, sgn):
47     # Returns new config by rotating input face of input config
48     # Rotation is clockwise if sgn == 1, counterclockwise if sgn == -1
49     assert face in (0, 1, 2)
50     assert sgn in (-1, 1)
51     if face is None:
52         return config
53     new_config = list(config)
54     if face == 0:
55         shift(new_config, 1 * sgn, [0, 1, 3, 2])
56         shift(new_config, 2 * sgn, [11, 10, 9, 8, 7, 6, 5, 4])
57     elif face == 1:
58         shift(new_config, 1 * sgn, [4, 5, 13, 12])
59         shift(new_config, 2 * sgn, [0, 2, 6, 14, 20, 22, 19, 11])
60     elif face == 2:
61         shift(new_config, 1 * sgn, [6, 7, 15, 14])
62         shift(new_config, 2 * sgn, [2, 3, 8, 16, 21, 20, 13, 5])
63     return ''.join(new_config)
64
65 def neighbors(config):
66     # Return neighbors of config
67     ns = []
68     for face in (0, 1, 2):
69         for sgn in (-1, 1):
70             ns.append(rotate(config, face, sgn))
71     return ns
72
73 def explore_frontier(frontier, parent, verbose=False):
74     # Explore frontier, adding new configs to parent and new_frontier
75     # Prints size of frontier if verbose is True
76     if verbose:
77         print('Exploring next frontier containing # configs: %s' % len(frontier))
78     new_frontier = []
79     for f in frontier:
80         for config in neighbors(f):
81             if config not in parent:
82                 parent[config] = f
83                 new_frontier.append(config)
84     return new_frontier
85
86 def path_to_config(config, parent):
87     # Return path of configurations from root of parent tree to config

```

```

88     path = [config]
89     while path[-1] is not None:
90         path.append(parent[path[-1]])
91     path.pop()
92     path.reverse()
93     return path
94
95 def moves_from_path(path):
96     # Given path of configurations, return list of moves relating them
97     # Returns None if any adjacent configs on path are not related by a move
98     moves = []
99     for i in range(1, len(path)):
100         move = None
101         for face in (0, 1, 2):
102             for sgn in (-1, 1):
103                 if rotate(path[i - 1], face, sgn) == path[i]:
104                     move = (face, sgn)
105                     moves.append(move)
106             if move is None:
107                 return None
108     return moves
109
110 def path_from_moves(config, moves):
111     # Return the path of configurations from input config applying input moves
112     path = [config]
113     for move in moves:
114         face, sgn = move
115         config = rotate(config, face, sgn)
116         path.append(config)
117     return path
118
119 def scramble(config, n):
120     # Returns new configuration by applying n random moves to config
121     from random import randint
122     for _ in range(n):
123         ns = neighbors(config)
124         i = randint(0, 2)
125         config = ns[i]
126     return config
127
128 def check(config, moves, verbose=False):
129     # Checks whether applying moves to config results in the solved config
130     if verbose:
131         print('Making %s moves from starting configuration:' % len(moves))
132     path = path_from_moves(config, moves)
133     if verbose:

```

```
134     print(config_str(config))
135     for i in range(1, len(path)):
136         face, sgn = moves[i - 1]
137         direction = 'clockwise'
138         if sgn == -1:
139             direction = 'counterclockwise'
140         if verbose:
141             print('Rotating face %s %s:' % (face, direction))
142             print(config_str(path[i]))
143     return path[-1] == SOLVED
144
145 def test(config):
146     print('Solving configuration:')
147     print(config_str(config))
148     moves = solve(config)
149     if moves is None:
150         print('Path to solved state not found... :(')
151         return
152     print('Path to solved state found!')
153     if check(config, moves):
154         print('Move sequence terminated at solved state!')
155     else:
156         print('Move sequence did not terminate at solved state... :(')
157
158 if __name__ == '__main__':
159     config = scramble(SOLVED, 100)
160     test(config)
```